

QUESTIONS 20 / 20 completed

Fibonacci

FIBONACCI

Write a program that prints the first n Fibonacci numbers using a function.

PROGRAM EDITOR

C



Run

Save

```
1 #include <stdio.h>
2
3 void fibonacci(int n) {
4     int a = 0, b = 1, c;
5
6     for (int i = 0; i < n; i++) {
7         printf("%d ", a);
8         c = a + b;
9         a = b;
10        b = c;
11    }
12 }
13
14 int main() {
15     int n;
16     scanf("%d", &n);
17
18     fibonacci(n);
19
20     return 0;
21 }
```

INPUT

5

QUESTIONS 20 / 20 completed

Factorial

FACTORIAL

Write a program to find the factorial of a number using recursion.

PROGRAM EDITOR

C



Run

Save

```
1 #include <stdio.h>
2
3 int factorial(int n)
4 {
5     if (n == 0 || n == 1)
6         return 1;
7     else
8         return n * factorial(n - 1);
9 }
10
11 int main()
12 {
13     int n;
14
15     scanf("%d", &n);
16
17     printf("%d", factorial(n));
18
19     return 0;
20 }
```

INPUT

5

QUESTIONS 20 / 20 completed

Fibonacci

FIBONACCI

Write a program to calculate the nth Fibonacci number using recursion.

PROGRAM EDITOR

C



Run

Save

```
1 #include<stdio.h>
2 int fibonacci(int n)
3 {
4     if(n==0)
5         return 0;
6     else if(n==1)
7         return 1;
8     else
9         return fibonacci(n-1)+fibonacci(n-2);
10 }
11 int main()
12 {
13     int terms;
14     scanf("%d",&terms);
15     printf("fibonacci sequence\n");
16     for(int i=0;i<terms;i++)
17         printf("%d ",fibonacci(i));
18     return 0;
19 }
```

INPUT

10

QUESTIONS 20 / 20 completed

Natural numbers ▾

NATURAL NUMBERS

Write a program to calculate the sum of the first n natural numbers using recursion.

PROGRAM EDITOR

C ▾

Run

Save

```
1 #include <stdio.h>
2
3 int sum(int n)
4 {
5     if (n == 0)
6         return 0;
7     else
8         return n + sum(n - 1);
9 }
10
11 int main()
12 {
13     int n;
14
15     scanf("%d", &n);
16
17     printf("%d", sum(n));
18
19     return 0;
20 }
```

INPUT

5

QUESTIONS 20 / 20 completed

GCD

GCD

Write a program to find the GCD of two numbers using recursion.

PROGRAM EDITOR

C



Run

Save

```
1 #include <stdio.h>
2
3 int gcd(int a, int b)
4 {
5     if (b == 0)
6         return a;
7     return gcd(b, a % b);
8 }
9
10 int main()
11 {
12     int a, b;
13
14     scanf("%d %d", &a, &b);
15
16     printf("%d", gcd(a, b));
17
18     return 0;
19 }
```

INPUT

48 18

QUESTIONS 20 / 20 completed

Power

POWER

Write a program to calculate the power of a number using recursion.

PROGRAM EDITOR

C



Run

Save

```
1 #include <stdio.h>
2
3 int power(int base, int exponent)
4 {
5     if (exponent == 0)
6         return 1;
7     else
8         return base * power(base, exponent - 1);
9 }
10
11 int main()
12 {
13     int base, exponent;
14
15     scanf("%d %d", &base, &exponent);
16
17     printf("%d", power(base, exponent));
18
19     return 0;
20 }
```

INPUT

2 5

QUESTIONS 20 / 20 completed

Reverse

REVERSE

Write a program to reverse a string using recursion.

PROGRAM EDITOR

C



Run

Save

INPUT

Min

```
1 #include <stdio.h>
2 #include <string.h>
3
4 void reverse(char str[], int index)
5 {
6     if (str[index] == '\0')
7         return;
8
9     reverse(str, index + 1);
10    printf("%c", str[index]);
11 }
12
13 int main()
14 {
15     char str[100];
16
17     scanf("%99s", str);
18
19     reverse(str, 0);
20
21     return 0;
```

QUESTIONS 20 / 20 completed

Palindrome

PALINDROME

Write a program to check if a given string is a palindrome using recursion.

PROGRAM EDITOR

C



Run

Save

```
1 #include <stdio.h>
2 #include <string.h>
3
4 int palindrome(char str[], int start, int end)
5 {
6     if (start >= end)
7         return 1;
8
9     if (str[start] != str[end])
10        return 0;
11
12    return palindrome(str, start + 1, end - 1);
13 }
14
15 int main()
16 {
17     char str[100];
18
19     scanf("%99s", str);
20
21     if (palindrome(str, 0, strlen(str) - 1))
```

INPUT

Madam

QUESTIONS 20 / 20 completed

Permutations

PERMUTATIONS

Write a program to generate all permutations of a given string using recursion.

PROGRAM EDITOR

C

Run

Save

```
1 #include <stdio.h>
2 #include <string.h>
3
4 void swap(char *a, char *b)
5 {
6     char temp = *a;
7     *a = *b;
8     *b = temp;
9 }
10
11 void permute(char str[], int left, int right)
12 {
13     int i;
14
15     if (left == right)
16     {
17         printf("%s\n", str);
18         return;
19     }
20
21     for (i = left; i <= right; i++)
```

INPUT

ABC

QUESTIONS 20 / 20 completed

Combinations

COMBINATIONS

Write a program to print all possible combinations of a given set of numbers using recursion.

PROGRAM EDITOR

C



Run

Save

```
1 #include <stdio.h>
2
3 void combinations(int a[], int n, int index)
4 {
5     int i;
6
7     if (index == n)
8     {
9         printf("{ ");
10        for (i = 0; i < n; i++)
11        {
12            if (a[i] != -1)
13                printf("%d ", a[i]);
14        }
15        printf("}\n");
16        return;
17    }
18
19    combinations(a, n, index + 1);
20
21    int temp = a[index];
```

INPUT

```
3
1 2 3
```

QUESTIONS 20 / 20 completed

Tower of Hanoi

TOWER OF HANOI

Write a program to solve the Tower of Hanoi problem using recursion.

PROGRAM EDITOR

C

Run

Save

```
1 #include <stdio.h>
2
3 void hanoi(int n, char source, char auxiliary, char destination)
4 {
5     if (n == 1)
6     {
7         printf("Move disk 1 from %c to %c\n", source, destination);
8         return;
9     }
10
11     hanoi(n - 1, source, destination, auxiliary);
12
13     printf("Move disk %d from %c to %c\n", n, source, destination);
14
15     hanoi(n - 1, auxiliary, source, destination);
16 }
17
18 int main()
19 {
20     int n;
21
```

INPUT

3

QUESTIONS 20 / 20 completed

Sum

SUM

Write a program to find the sum of digits of a given number using recursion.

PROGRAM EDITOR

C



Run

Save

```
1 #include <stdio.h>
2
3 int sumDigits(int n)
4 {
5     if (n == 0)
6         return 0;
7
8     return (n % 10) + sumDigits(n / 10);
9 }
10
11 int main()
12 {
13     int n;
14
15     scanf("%d", &n);
16
17     printf("%d", sumDigits(n));
18
19     return 0;
20 }
```

INPUT

12345

QUESTIONS 20 / 20 completed

Geometric series ▾

GEOMETRIC SERIES

Write a program to print the first n terms of the geometric series using recursion.

PROGRAM EDITOR

C ▾

Run

Save

```
1 #include <stdio.h>
2
3 void geometric(float a, float r, int n)
4 {
5     if (n == 0)
6         return;
7
8     printf("%.2f ", a);
9
10    geometric(a * r, r, n - 1);
11 }
12
13 int main()
14 {
15     float a, r;
16     int n;
17
18     scanf("%f %f %d", &a, &r, &n);
19
20     geometric(a, r, n);
21
```

INPUT

2 3 5

QUESTIONS 20 / 20 completed

Arithmetic series ▾

ARITHMETIC SERIES

Write a program to find the nth term of the arithmetic series using recursion.

PROGRAM EDITOR

C ▾

Run

Save

```
1 #include <stdio.h>
2
3 int nthTerm(int a, int d, int n)
4 {
5     if (n == 1)
6         return a;
7
8     return d + nthTerm(a, d, n - 1);
9 }
10
11 int main()
12 {
13     int a, d, n;
14
15     scanf("%d %d %d", &a, &d, &n);
16
17     printf("%d", nthTerm(a, d, n));
18
19     return 0;
20 }
```

INPUT

2 3 5

QUESTIONS 20 / 20 completed

Even

EVEN

Write a program to calculate the sum of the first n even numbers using recursion.

PROGRAM EDITOR

C



Run

Save

```
1 #include <stdio.h>
2
3 int sumOfEvens(int n) {
4     if (n <= 0) {
5         return 0;
6     }
7     return (2 * n) + sumOfEvens(n - 1);
8 }
9
10 int main() {
11     int n;
12
13     scanf("%d", &n);
14
15     printf("%d", sumOfEvens(n));
16
17     return 0;
18 }
```

INPUT

5

QUESTIONS 20 / 20 completed

D to B

D TO B

Write a program to find the binary equivalent of a given decimal number using recursion.

PROGRAM EDITOR

C



Run

Save

```
1 #include <stdio.h>
2
3 void toBinary(int n) {
4     if (n == 0) return;
5     toBinary(n / 2);
6     printf("%d", n % 2);
7 }
8
9 int main() {
10     int num = 13;
11     if (num == 0) printf("0");
12     else toBinary(num);
13     printf("\n");
14     return 0;
15 }
```

INPUT

4

QUESTIONS 20 / 20 completed

Length

LENGTH

Write a program to find the length of a given string using recursion.

PROGRAM EDITOR

C



Run

Save

```
1 #include <stdio.h>
2
3 int stringLength(char *str) {
4     if (*str == '\0') return 0;
5     return 1 + stringLength(str + 1);
6 }
7
8 int main() {
9     char str[] = "Hello";
10    printf("%d\n", stringLength(str));
11    return 0;
12 }
```

INPUT

Hello

QUESTIONS 20 / 20 completed

Occurrences

OCCURRENCES

Write a program to find the number of occurrences of a given character in a string using recursion.

PROGRAM EDITOR

C



Run

Save

```
1 #include <stdio.h>
2
3 int countOccurrences(char *str, char ch) {
4     if (*str == '\0') return 0;
5     return (*str == ch) + countOccurrences(str + 1, ch);
6 }
7
8 int main() {
9     char str[] = "hello world";
10    char ch = 'l';
11    printf("%d\n", countOccurrences(str, ch));
12    return 0;
13 }
```

QUESTIONS 20 / 20 completed

Sum Array



SUM ARRAY

Write a program to find the sum of the elements in a given array using recursion.

PROGRAM EDITOR

C



Run

Save

```
1 #include <stdio.h>
2
3 int sumArray(int arr[], int n) {
4     if (n <= 0) return 0;
5     return arr[n - 1] + sumArray(arr, n - 1);
6 }
7
8 int main() {
9     int arr[] = {1, 2, 3, 4, 5};
10    int n = sizeof(arr) / sizeof(arr[0]);
11    printf("%d\n", sumArray(arr, n));
12    return 0;
13 }
```

QUESTIONS 20 / 20 completed

Perfect

PERFECT

Write a program to check if a given number is a perfect square using recursion.

PROGRAM EDITOR

C

Run

Save

```
1 #include <stdio.h>
2
3 int isPerfectSquare(int n, int i) {
4     if (i * i == n) return 1;
5     if (i * i > n) return 0;
6     return isPerfectSquare(n, i + 1);
7 }
8
9 int main() {
10     int num = 25;
11     if (num < 0) {
12         printf("Not a perfect square\n");
13     } else if (isPerfectSquare(num, 0)) {
14         printf("Perfect square\n");
15     } else {
16         printf("Not a perfect square\n");
17     }
18     return 0;
19 }
```